

2026-06-10-B-model-access

Workstream B — Model-Access Refinement: API-Key Auto-Recognition + Dynamic Working-Model Exposure via LLMsVerifier

Revision: 1 **Created:** 2026-06-10 **Last modified:** 2026-06-10 **Status:** draft **Maintainer:**
Workstream-B analysis subagent (READ-ONLY planning pass) **Scope:** Evidence-cited gap
analysis. No code changed. All paths absolute. ABSENT items flagged.

Goal restated (from the request): *“For every provider, if an API key is provided OR recognized from .env / .api_keys / shell-exported, that provider’s WORKING models become available — everything dynamically obtained + validated + verified via LLMsVerifier.”*

Table of contents

- [1. API-key loading — current state + GAP](#)
 - [2. LLMsVerifier integration — real vs stubbed](#)
 - [3. Provider inventory — dynamic vs hardcoded + key-gating](#)
 - [4. Design — key-recognized → verifier-validates → only-working-exposed](#)
 - [5. Deliverables the request demands](#)
 - [Executive summary + top 5 gaps](#)
-

1. API-key loading

1.1 What loads keys today (cited)

Four independent, partly-overlapping mechanisms exist; none of them implement the request goal end-to-end.

(a) Viper AutomaticEnv + explicit BindEnv (HelixCode primary path). /Volumes/T7/Projects/HelixCode/helix_code/internal/config/config.go - L307–309
v.AutomaticEnv() + v.SetEnvPrefix("HELIX"). - L337–345 per-provider key
bindings into config keys, e.g.
v.BindEnv("verifier.providers.openai.api_key", "OPENAI_API_KEY")
... through openrouter (openai, anthropic, gemini, deepseek, groq, mistral, xai, openrouter —
8 providers). - These map shell-exported env vars into
cfg.Verifier.Providers[name].APIKey.

(b) Per-provider constructor os.Getenv fallback (the de-facto runtime path). Every cloud
provider constructor reads its env var directly if config.APIKey is empty: - /Volumes/T7/Projects/HelixCode/helix_code/internal/llm/openai_provider.go L45–
51 (apiKey = os.Getenv("OPENAI_API_KEY"); hard error if still empty). - /
Volumes/T7/Projects/HelixCode/helix_code/internal/llm/
anthropic_provider.go L153 (apiKey =

os.Getenv("ANTHROPIC_API_KEY")), plus ANTHROPIC_BASE_URL L169. - azure_provider.go L285 (AZURE_OPENAI_API_KEY), bedrock_provider.go L272/276 (AWS_REGION/AWS_DEFAULT_REGION), and gemini/groq/deepseek/mistral/openrouter/xai mirror the same if apiKey == "" { ... os.Getenv(...) } pattern (grep evidence: §3.3 table).

(c) .env / api_keys.sh loader — **EXISTS but is UNWIRED (critical gap)**. /Volumes/T7/Projects/HelixCode/helix_code/internal/secrets/loader.go - L30 func LoadAPIKeys() error — reads \$HOME/api_keys.sh first (L33–37, parses export VAR=value lines, loadFromShell), else walks up from cwd to find a .env (findEnvFile L119–129, loadFromEnv L84–106), applying values via os.Setenv. - Real file I/O, no simulation; CONST-042-aware (never logs values). - **GAP**: grep across helix_code/cmd, helix_code/internal/server, helix_code/applications returns ZERO production call-sites. The ONLY references are unit tests (/Volumes/T7/Projects/HelixCode/helix_code/internal/secrets/loader_test.go, .../translator_test.go L130). So HelixCode does **not** auto-load .env or api_keys.sh at startup — it depends entirely on the operator having already shell-exported the keys (mechanisms a + b). This directly fails the request clause “recognized from .env / .api_keys”. (Note: this is exactly the §11.4.124 unwired-code situation — investigate-before-anything; the fix here is to WIRE it, not remove it.)

(d) .env.example (repo root). /Volumes/T7/Projects/HelixCode/.env.example L42–49 lists the 8 cloud key vars (OPENAI_API_KEY, ANTHROPIC_API_KEY, GEMINI_API_KEY, DEEPSEEK_API_KEY, GROQ_API_KEY, MISTRAL_API_KEY, XAI_API_KEY, OPENROUTER_API_KEY), all empty; L30–37 the verifier config block (HELIX_VERIFIER_ENABLED=false default, HELIX_VERIFIER_ENDPOINT=http://localhost:8081, HELIX_VERIFIER_MIN_SCORE=6.0, HELIX_MODELS_DEV_ENDPOINT=https://api.models.dev); L54–55 local provider hosts. There is no .api_keys filename literally — the request’s “.api_keys” maps to loader.go’s \$HOME/api_keys.sh shell-recipe file.

1.2 helix_agent reference implementation (the design template)

/Volumes/T7/Projects/HelixCode/submodules/helix_agent/internal/verifier/provider_types.go - L348–349 var SupportedProviders = map[string]*ProviderTypeInfo{...} declares **every** provider with a per-provider EnvVars list (multiple aliases each), e.g. L357 anthropic {"ANTHROPIC_API_KEY", "CLAUDE_API_KEY"}, L381 gemini {"GEMINI_API_KEY", "GOOGLE_API_KEY", "ApiKey_Gemini"}, L552 xai {"XAI_API_KEY", "GROK_API_KEY"}, plus cerebras, perplexity, cohere, ai21, together, fireworks, zai, qwen, openrouter, ollama, llama.cpp (20+ providers).

/Volumes/T7/Projects/HelixCode/submodules/helix_agent/internal/verifier/startup.go - L347–348 unsupportedScanner := api_keys.NewEnvVarScanner(); unsupportedScanner.ScanEnvForUnsupportedKeys() — records env-detected keys that don’t map to a known provider. - L389–405 the core loop: for each providerType in SupportedProviders, for each envVar in info.EnvVars: apiKey := os.Getenv(envVar); if apiKey != "" && !isPlaceholder(apiKey) → **try dynamic model discovery first** (sv.DiscoverModels(ctx, providerType, apiKey)), fall back to info.Models static list only if discovery fails. - L342/360–364 + L1377/1401 integrate a faulty-key registry (api_keys.ReadFaultyAPIKeys, GetProviderAPIKeyName, WriteFaultyAPIKey) so providers with revoked/failing keys are de-prioritised.

The `api_keys` package lives in the `LLMsVerifier` submodule: `/Volumes/T7/Projects/HelixCode/submodules/llms_verifier/llm-verifier/api_keys/{env_scanner.go,manager.go,priority.go}` (imported in `helix_agent` as `digital.vasic.llmsverifier/api_keys`, `startup.go` L26).

This is exactly the “key recognized → discover → expose” flow the request wants. HelixCode does not have it; helix_agent does. Per §11.4.74 the design should reuse/extend this, not re-invent it.

2. LLMsVerifier integration

2.1 Consumption is REAL (not stubbed)

`/Volumes/T7/Projects/HelixCode/helix_code/internal/verifier/client.go` - Real `stdlib net/http` REST client. `GetModels` L72–94 → GET `{baseUrl}/api/models`, JSON-decodes `[]*VerifiedModel`. Also `Health` L47 (`/api/health`), `GetModelByID` L97, `GetProviderScores` L126 (`/api/scores`), `VerifyModel` L151 (POST `/api/models/{id}/verify`), `GetPricing` L177, `GetLimits` L202. `setAuthHeader` L227 adds `Authorization: Bearer` when an API key is set. No simulation.

`/Volumes/T7/Projects/HelixCode/helix_code/internal/verifier/adapter.go` - `GetVerifiedModels` L183–224: gated on `IsEnabled()` (L184) + circuit-breaker `health.AllowRequest()` (L187); cache-first (L192); live `client.GetModels` (L199); on failure → stale cache (L205) → `getFallbackModels()` (L210, hardcoded `FallbackModels`, marked `Source="fallback"`, returns `ErrUsingFallback`). - `filterByProviderConfig` L277–289: drops models whose `config.Providers[m.Provider].Enabled == false`. - `refreshScores` L256–274 populates `modelScores/modelCodeScores/modelRelScores/providerScores` from the live list.

`/Volumes/T7/Projects/HelixCode/helix_code/internal/verifier/types.go` - `VerifiedModel` L24: carries `Verified bool` (L31), `VerificationStatus string` (L32: `pending|verified|failed|rate_limited`), `OverallScore float64` (L48), `LastVerified` (L54). These are the fields that determine “working”.

2.2 How “working models” are determined today

There is **no single “working models” funnel**. Two disjoint exposure paths exist:

Path A — verifier-authoritative (CLI + server). `/Volumes/T7/Projects/HelixCode/helix_code/cmd/cli/main.go` `handleListModels` L1355–1390: 1. L1360 if `verifierAdapter != nil && IsEnabled()` → `GetVerifiedModels(ctx)` → print all (L1363). 2. L1371 else if `c.llmProvider != nil` → `c.llmProvider.GetModels()` (single configured provider). 3. L1387 else → `verifier.FallbackModels` (constitutional hardcoded fallback). `printVerifiedModels` L1392–1414 shows score + `Verified/pending/failed/rate_limited` status — but it **prints failed/pending models too** (status string only; no filtering). Server mirror: `/Volumes/T7/Projects/HelixCode/helix_code/internal/server/handlers.go` L1049/L1140/L1206 all call `s.verifierResult.Adapter.GetVerifiedModels(ctx)`.

Path B — provider GetModels() (capability enrichment). /Volumes/T7/Projects/HelixCode/helix_code/internal/llm/verifier_bridge.go
EnrichModelInfo L26–67: when verifier enabled, augments a provider’s ModelInfo with verifier_score, code/reliability caps, tool-support heuristic (score > 7.0 → SupportsTools). Falls back to inferCapabilitiesFromModelID (string heuristics) when verifier off (L65). Bridge VerifierModelSource.FetchModels(verifier_integration.go L24–33) converts VerifiedModel → ModelInfo but is not the path the CLI/server actually use for listing.

2.3 GAP — no key-presence gating in the verifier path

grep over helix_code/internal/verifier/*.go (non-test) finds APIKey only in adapter/bootstrap config plumbing — **the verifier path never checks whether the operator holds a key for a given provider.** GetVerifiedModels returns the verifier’s ENTIRE catalog (every provider the verifier knows), filtered only by config.Providers[*].Enabled (an explicit, manually-maintained flag — adapter.go L283), not by “does this user have a usable key for provider X”. So today a user with only ANTHROPIC_API_KEY set still sees OpenAI, Gemini, etc. listed as “available.” That is the opposite of the request goal.

2.4 Score threshold exists but is not applied as a “working” filter

adapter.go GetMinAcceptableScore L175–180 (default 6.0, from HELIX_VERIFIER_MIN_SCORE) is defined but **not invoked** inside GetVerifiedModels / filterByProviderConfig — there is no OverallScore >= min filter and no Verified == true filter in the listing path. “Working model” (verified + scored + key-present) is therefore not computed anywhere.

3. Provider inventory

All cloud provider GetModels() return a stored provider.models slice populated at construction by initializeModels(); the interface is synchronous GetModels() []ModelInfo (no ctx/err) — /Volumes/T7/Projects/HelixCode/helix_code/internal/llm/provider.go defines it.

Provider	File (helix_code/internal/llm/)	Model source	Hardcoded? (CONSENSUS risk)
OpenAI	openai_provider.go	initializeModels L201–239	HARDCODED (gpt-turbo, gpt-4, gpt-3.5-...
Anthropic	anthropic_provider.go	models built L205–287	HARDCODED (claude-sonnet/opus, 3-7, 3-5-sonnet/haiku)
Gemini	gemini_provider.go	GetModels L299; init	likely HARDCODED (not present)
Groq	groq_provider.go	L190	likely HARDCODED
DeepSeek	deepseek_provider.go		HARDCODED

Provider	File (helix_code/internal/llm/)	Model source	Hardcoded? (CONST-036 risk)
		return dp.models L80; init L177	
Mistral	mistral_provider.go	return mp.models L66; init L165	HARDCODED
xAI	xai_provider.go	L72; init L199	likely HARDCODED
OpenRouter	openrouter_provider.go	fetchCatalog L233–295, initializeModels L297+	DYNAMIC — live models (L236), cur fallback only if unre
Ollama	ollama_provider.go	GetModels L225	DYNAMIC — /ap (local)
Llama.cpp	llamacpp_provider.go	L60–96	DYNAMIC-ish (retur []ModelInfo{ } v
Azure	azure_provider.go	L463	HARDCODED/depl map (AZURE_DEPLOYME L303)
Bedrock	bedrock_provider.go	L421	HARDCODED
VertexAI	vertexai_provider.go	L451	HARDCODED
Copilot	copilot_provider.go	L191; init L318	HARDCODED
OpenAI-compatible	openai_compatible_provider.go	L168	config-driven
KoboldAI	koboldai_provider.go	L118	local
Qwen	qwen_provider.go	L117; init L245	HARDCODED + OA (qwen_oauth.go)

Notes: - **CONST-036 violations (hardcoded model lists in production)**: OpenAI, Anthropic, DeepSeek, Mistral (confirmed by reading the literal slices), plus very-likely Gemini, Groq, xAI, Azure, Bedrock, VertexAI, Copilot, Qwen. OpenRouter (round-41 fix, L297–311 comment) and Ollama/Llama.cpp are already dynamic — these are the templates. - `factory.go` `NewProvider` (L9 switch) constructs ONE provider by `config.Type`; there is no “construct every provider for which a key is present” enumerator anywhere. - `auto_llm_manager.go` is a **local-LLM lifecycle manager** (clone/build/start Ollama/ llama.cpp etc.) — unrelated to cloud key→model exposure; do not conflate. - `model_manager.go` `getAvailableModels` L202–208 just flattens an in-memory `modelRegistry`; `SetVerifierAdapter` L132 exists but the registry is populated from provider `GetModels()` (the hardcoded lists), not from the verifier catalog.

4. Design

4.1 Target flow: key recognized → verifier validates → only working models exposed

```
startup
├ secrets.LoadAPIKeys() [WIRE THIS — currently unwired]
  └ reads $HOME/api_keys.sh, then .env (walk-up), os.Setenv each
├ KeyRecognition (NEW, reuse helix_agent SupportedProviders.EnvVars + a
  └ for each provider: present = any(os.Getenv(envVarAlias)) != "" &&
    → set of CONFIGURED providers
├ verifier.Adapter.GetVerifiedModels(ctx) [REAL HTTP, exists]
├ WorkingModelFilter (NEW)
  └ keep model m iff present[m.Provider] (key-presence ga
    AND m.Verified == true (verifier-valida
    AND m.VerificationStatus == "verified"
    AND m.OverallScore >= GetMinAcceptableScore() [exi
└ expose (CLI handleListModels / server /api/models / model_manager reg
```

4.2 Concrete wiring points (where to change)

1. **Wire the key loader.** Call `secrets.LoadAPIKeys()` (loader.go L30) at the start of /
Volumes/T7/Projects/HelixCode/helix_code/cmd/server/ and /
Volumes/T7/Projects/HelixCode/helix_code/cmd/cli/main.go
main() BEFORE config load (so .env/api_keys.sh populate os.Environ()
ahead of viper AutomaticEnv and the provider os.Getenv fallbacks). Currently never
called (§1.1c).
2. **Add provider key-recognition table to HelixCode** (or reuse helix_agent's
SupportedProviders via the catalogue per §11.4.74/§11.4.51 — it lives in
submodules/helix_agent/internal/verifier/provider_types.go
L349; the api_keys scanner lives in submodules/llms_verifier/llm-
verifier/api_keys/). HelixCode's own config.go only binds 8 single-alias keys
(L337–345) — narrower than helix_agent's multi-alias map; widen it.
3. **Add the working-model filter** inside adapter.GetVerifiedModels (/Volumes/
T7/Projects/HelixCode/helix_code/internal/verifier/adapter.go
L223 return site) OR as a new method GetWorkingModels(ctx,
presentProviders) that applies the key-presence + Verified +
VerificationStatus=="verified" + OverallScore>=min (L175
GetMinAcceptableScore) predicate. The min-score threshold is loaded but unused
— apply it here.
4. **Make filterByProviderConfig key-aware** (adapter.go L277): today it only
honours a manual Enabled flag; auto-derive Enabled from key-presence instead of (or
in addition to) the static config flag.
5. **Update exposure call-sites** to the new working-model path: CLI handleListModels
(cmd/cli/main.go L1361 — and stop printing failed/pending rows as available),
server handlers (internal/server/handlers.go L1049/L1140/L1206), and
model_manager registry population (model_manager.go L132 SetVerifierAdapter
+ L202 getAvailableModels).
6. **Convert remaining hardcoded provider initializeModels to dynamic**
(CONST-036): OpenAI/Anthropic/Gemini/etc. should fetch their live catalog (mirror
OpenRouter fetchCatalog L233 / Ollama / api / tags) OR defer entirely to the
verifier catalog, so nothing the user sees is a stale literal. This also closes the BLUFF-002
regression risk.

7. **Verifier-disabled degradation:** when `HELIX_VERIFIER_ENABLED=false` (default in `.env.example` L30), define the honest behaviour — either require enabling for “working models”, or fall back to per-provider live `GetModels()` for key-present providers only (never the hardcoded fallback list presented as “working” — that would be a `CONST-035/ §11.4 PASS-bluff`).

4.3 Anti-bluff constraints binding this work

- `CONST-036/037/039/040`: verifier is the single source of truth; no hardcoded model lists; all 10 mandated providers; capability flags from `VerificationResult`.
 - `§11.4.124: secrets.LoadAPIKeys` is unwired-but-tested code — WIRE it (investigate-before- remove already satisfied; it has unit tests and a clear intended call-site per its doc-comment).
 - `CONST-042/§12.1 + CONST-053`: keys via `.env/api_keys.sh` (mode 0600, gitignored), never logged (loader.go already compliant), never committed.
 - `§11.4.107/§11.4.69 + CONST-050`: tests beyond unit MUST hit a real verifier + real provider endpoints; a model shown as “working” MUST be backed by captured runtime evidence (real generate call returns real output), not metadata-only.
-

5. Deliverables

What the request demands once implemented (per `CONST-047/048/050/§11.4.83/§11.4.135`):

- **Code:** key-recognition table (multi-alias, all `CONST-039` providers) + `LoadAPIKeys` wiring + `GetWorkingModels` filter (`key^Verified^status^min-score`) + dynamic `initializeModels` conversions + updated `CLI/server/model_manager` exposure.
 - **Tests (4-layer, §11.4.4b):** unit (filter predicate, key-recognition, placeholder rejection — mocks OK here only); integration - `tags=integration` against REAL verifier (make `test-verifier-integration`) + REAL provider key (no mocks, `CONST-050`); E2E that sets only one provider’s key and asserts ONLY that provider’s verified models list + a real `generate` returns real output (`§11.4.107` liveness, not metadata); stress/chaos per `§11.4.85` (verifier unreachable → honest degradation, not fallback-as-working); paired `§1.1` mutation (strip the key-presence check → a no-key provider’s models leak → test FAILs); standing regression guard per `§11.4.135` (`RED_MODE` polarity).
 - **Challenges:** HelixQA + `challenges/` bank covering “set `ANTHROPIC_API_KEY` only → `cli --list-models` shows only Anthropic verified models → generate works” with captured bidirectional transcript under `docs/qa/<run-id>/` (`§11.4.83`).
 - **Docs/guides (CONST-062/066, always-sync .md+.html+.pdf):** update `.env.example` (document `api_keys.sh` + `.env` auto-load + per-provider aliases), a `docs/guides/model-access.md` user manual (how key-presence → working models works, verifier-disabled behaviour), `docs/ARCHITECTURE.md` flow update, and the relevant `Status/Status_Summary` + `Issues/Fixed` entries (`ATM-NNN` per `§11.4.54`).
-

Executive summary

1. **Key loading is fragmented and the `.env/api_keys.sh` path is DEAD.**
`secrets.LoadAPIKeys` (`helix_code/internal/secrets/loader.go` L30) reads `$HOME/api_keys.sh` then a walked-up `.env`, but it is **never called** outside unit tests — production relies solely on shell-exported env (`viper.BindEnv` `config.go` L337–

- 345 + per-constructor `os.Getenv()`). The request's "recognized from `.env/.api_keys`" is therefore unmet.
2. **LLMsVerifier consumption is REAL, not stubbed** — `verifier/client.go` does live `net/http` to `/api/models` etc.; `adapter.GetVerifiedModels` is `cache→live→stale→fallback` with circuit-breaking.
 3. **No key-presence gating anywhere in the model-exposure path.** `GetVerifiedModels` returns the verifier's whole catalog filtered only by a manual `Providers[*].Enabled` flag (`adapter.go` L277–289); a user with one key still sees every provider.
 4. **The "working model" predicate is not computed.** `VerifiedModel.Verified/VerificationStatus/OverallScore` exist (`types.go` L31/32/48) and `GetMinAcceptableScore` (L175) is loaded from `HELIX_VERIFIER_MIN_SCORE`, but no listing path filters on `verified^status^min-score`; CLI even prints `failed/pending` rows.
 5. **Several provider `GetModels()` are hardcoded (CONST-036 risk):** OpenAI (L203–232), Anthropic (L205–287), DeepSeek, Mistral confirmed; Gemini/Groq/xAI/Azure/Bedrock/VertexAI/Copilot/Qwen likely. Only OpenRouter (live `fetchCatalog`) + Ollama/Llama.cpp are dynamic and serve as the conversion template.
 6. **The exact target design already exists in `helix_agent`** — `submodules/helix_agent/internal/verifier/{provider_types.go` L349 `SupportedProviders+EnvVars`, `startup.go` L389–405 `per-key→DiscoverModels` + `submodules/llms_verifier/.../api_keys/`. Per §11.4.74 reuse/extend rather than re-implement.
 7. Min-score threshold + verifier-disabled degradation are undefined behaviours that must be specified to avoid presenting hardcoded fallbacks as "working" (a §11.4 PASS-bluff).
 8. Provider construction has no "build every key-present provider" enumerator — `factory.go` builds ONE provider by `config.Type`.
 9. Closing this needs 4-layer tests + Challenges + always-sync docs per CONST-047/048/050/ §11.4.83/§11.4.135, with captured liveness evidence (§11.4.107).
 10. All findings are READ-ONLY; no code was modified in this pass.

Top 5 concrete gaps

1. `secrets.LoadAPIKeys` is **never invoked at startup** → `.env/api_keys.sh` auto-recognition does not happen. Fix: call it first in `cmd/server + cmd/cli/main.go` `main()`. (`helix_code/internal/secrets/loader.go` L30; zero production call-sites.)
2. **No key-presence → provider-availability gate.** The verifier path (`adapter.GetVerifiedModels` L183; CLI `handleListModel` L1361) lists the full catalog regardless of which keys the operator holds. Need a key-recognition table (reuse `helix_agent.SupportedProviders.EnvVars`) + a present-providers filter.
3. **No "working model" filter** combining `key-present ^ Verified==true ^ VerificationStatus=="verified" ^ OverallScore >= GetMinAcceptableScore()`. The threshold (`adapter.go` L175) is loaded but unused; CLI prints `failed/pending` models.
4. **Hardcoded provider model lists** (OpenAI L203, Anthropic L205, DeepSeek, Mistral, + ~8 more) violate CONST-036; convert to dynamic (OpenRouter `fetchCatalog` L233 / Ollama `/api/tags` templates) or defer to the verifier catalog.
5. **Undefined verifier-disabled / cold behaviour** (`HELIX_VERIFIER_ENABLED=false` default): the current fallback to `verifier.FallbackModels` (CLI L1387) presents a constitutional hardcoded list as available — must be specified so it is never shown as "working" without verifier validation (else §11.4/CONST-035 PASS-bluff).